Challenge 3 - Confidential Hospital Records

192521079

Problem Statement

A hospital wants to protect patient details such as patient ID, name, diagnosis, and prescription. Develop an application that encrypts the patient information before storing it and decrypts it only for authorized users, maintaining the confidentiality of patient data.

Objective

Build a working application that:

1. **Encrypts** patient records (name, diagnosis, prescription) before they are ever written to storage — so data at rest is never plaintext.
2. **Decrypts** records only after successfully verifying the identity of the requesting user (authentication).
3. **Restricts** who is allowed to see decrypted sensitive fields, even among authenticated users (authorization / role-based access control).
4. Demonstrates that unauthorized access attempts — wrong password, disallowed role, or unknown user — are correctly rejected.

Design and Security Approach

Requirement	Mechanism used	Why
Confidentiality of stored data	**Fernet symmetric encryption** (AES-128-CBC + HMAC-SHA256, from Python's `cryptography` library)	Fernet both encrypts and authenticates each record, so data at rest is unreadable without the key *and* tamper-evident
Key management	Symmetric key generated once and stored separately from the database file	Keeps the key logically separated from the encrypted data (in production this would be a hardware security module or key-management service, not a flat file)
User authentication	**PBKDF2-HMAC-SHA256** password hashing with a random salt per user, 100,000 iterations	Never stores plaintext passwords; salted, iterated hashing resists rainbow-table and brute-force attacks
Authorization	Role field per user (`doctor` / `admin` vs `receptionist`) checked *before* decryption is attempted	Enforces least privilege — even a correctly authenticated user cannot view sensitive medical data unless their role permits it
Integrity	Fernet's built-in HMAC verification	Detects if stored ciphertext has been tampered with; decryption fails safely rather than returning corrupted data

Data flow:

```
add_patient_record() → JSON payload → Fernet.encrypt() → stored as ciphertext in patient_db.json
view_patient_record() → authenticate(user, pwd) → check role → Fernet.decrypt() → plaintext returned only on success
```

Only the `patient_id` is kept as plaintext (used as a lookup key); all clinical fields (name, diagnosis,

prescription) are encrypted before storage, satisfying the requirement to protect exactly the sensitive fields named in the problem statement.

Full Application Code

```
    """
    Confidential Hospital Records — Encrypted Patient Data Management
    ------------------------------------------------------------------
    Encrypts patient details (ID, name, diagnosis, prescription) before
    storing them, and decrypts them only for authenticated, authorized
    users. Uses:
        - Fernet (AES-128 in CBC mode + HMAC-SHA256) for symmetric
          encryption/authentication of patient records (confidentiality
          + integrity).
        - PBKDF2-HMAC-SHA256 password hashing for user authentication
          (no plaintext passwords stored).
        - Role-based authorization (only users in the "doctor"/"admin"
          role may decrypt records).
    """

    import json
    import os
    import hashlib
    import hmac
    import base64
    from datetime import datetime
    from cryptography.fernet import Fernet

    DB_FILE = "patient_db.json"
    USER_FILE = "users.json"
    KEY_FILE = "secret.key"


    # ------------------------------------------------------------------
    # 1. Key management
    # ------------------------------------------------------------------
    def load_or_create_key():
        """Load the symmetric encryption key, generating one on first run.
        In production this key would be stored in a hardware security
        module / key vault, not a flat file."""
        if os.path.exists(KEY_FILE):
            with open(KEY_FILE, "rb") as f:
                return f.read()
        key = Fernet.generate_key()
        with open(KEY_FILE, "wb") as f:
            f.write(key)
        return key


    # ------------------------------------------------------------------
    # 2. User authentication (salted password hashing)
    # ------------------------------------------------------------------
    def hash_password(password, salt=None):
        if salt is None:
            salt = os.urandom(16)
        pwd_hash = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100_000)
        return base64.b64encode(salt).decode(), base64.b64encode(pwd_hash).decode()
```

```
def verify_password(password, salt_b64, hash_b64):
    salt = base64.b64decode(salt_b64)
    expected = base64.b64decode(hash_b64)
    pwd_hash = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100_000)
    return hmac.compare_digest(pwd_hash, expected)


def load_users():
    if not os.path.exists(USER_FILE):
        return {}
    with open(USER_FILE, "r") as f:
        return json.load(f)


def save_users(users):
    with open(USER_FILE, "w") as f:
        json.dump(users, f, indent=2)


def register_user(username, password, role):
    """role: 'doctor'/'admin' (authorized to decrypt) or 'receptionist'
    (authorized to add records but NOT to view diagnosis/prescription)."""
    users = load_users()
    salt, pwd_hash = hash_password(password)
    users[username] = {"salt": salt, "hash": pwd_hash, "role": role}
    save_users(users)


def authenticate(username, password):
    users = load_users()
    if username not in users:
        return None
    record = users[username]
    if verify_password(password, record["salt"], record["hash"]):
        return record["role"]
    return None


# ---------------------------------------------------------------------
# 3. Patient record encryption / storage
# ---------------------------------------------------------------------
def load_db():
    if not os.path.exists(DB_FILE):
        return []
    with open(DB_FILE, "r") as f:
        return json.load(f)


def save_db(records):
    with open(DB_FILE, "w") as f:
        json.dump(records, f, indent=2)


def add_patient_record(fernet, patient_id, name, diagnosis, prescription):
    """Encrypts the sensitive fields before writing to storage.
    Patient ID is kept as a plaintext lookup key (a hospital may choose
    to encrypt this too, at the cost of losing indexable search)."""
```

```
            payload = {
                "name": name,
                "diagnosis": diagnosis,
                "prescription": prescription,
                "timestamp": datetime.utcnow().isoformat(),
            }
            plaintext = json.dumps(payload).encode()
            ciphertext = fernet.encrypt(plaintext)

            records = load_db()
            records.append({
                "patient_id": patient_id,
                "encrypted_data": ciphertext.decode(),
            })
            save_db(records)
            print(f"[STORED] Patient {patient_id} added — data encrypted at rest.")


        def view_patient_record(fernet, username, password, patient_id):
            """Only returns decrypted data if the user authenticates successfully
            AND holds an authorized role. Unauthorized users never even reach
            the decryption step."""
            role = authenticate(username, password)
            if role is None:
                print(f"[DENIED] Authentication failed for '{username}'.")
                return None
            if role not in ("doctor", "admin"):
                print(f"[DENIED] User '{username}' (role: {role}) is not authorized "
                      f"to view decrypted diagnosis/prescription data.")
                return None

            records = load_db()
            match = next((r for r in records if r["patient_id"] == patient_id), None)
            if not match:
                print(f"[NOT FOUND] No record for patient {patient_id}.")
                return None

            try:
                plaintext = fernet.decrypt(match["encrypted_data"].encode())
                data = json.loads(plaintext)
                print(f"[AUTHORIZED ACCESS] {username} ({role}) decrypted record "
                      f"for patient {patient_id}.")
                return data
            except Exception:
                print("[ERROR] Decryption failed — data may have been tampered with.")
                return None


        def show_raw_stored_record(patient_id):
            """Demonstrates that data at rest is genuinely ciphertext —
            unreadable without the key, even by someone with direct file access."""
            records = load_db()
            match = next((r for r in records if r["patient_id"] == patient_id), None)
            if match:
                print(f"[RAW DB CONTENT] patient_id={patient_id} -> "
                      f"{match['encrypted_data'][:60]}...  (truncated ciphertext)")


        # ---------------------------------------------------------------
```

```
        # 4. Demonstration
        # ------------------------------------------------------------------
        if __name__ == "__main__":
            # Clean slate for repeatable demo runs
            for f in (DB_FILE, USER_FILE, KEY_FILE):
                if os.path.exists(f):
                    os.remove(f)

            key = load_or_create_key()
            fernet = Fernet(key)

            print("=== Setting up users ===")
            register_user("dr_asha", "SecurePass#1", role="doctor")
            register_user("front_desk", "Reception#22", role="receptionist")
            print("Registered: dr_asha (doctor), front_desk (receptionist)\n")

            print("=== Adding patient records (encrypted before storage) ===")
            add_patient_record(fernet, "P1001", "Ramesh Kumar",
                               "Type 2 Diabetes", "Metformin 500mg twice daily")
            add_patient_record(fernet, "P1002", "Sunita Rao",
                               "Hypertension", "Amlodipine 5mg once daily")
            print()

            show_raw_stored_record("P1001")
            print()

            print("=== Access attempt 1: authorized doctor ===")
            data = view_patient_record(fernet, "dr_asha", "SecurePass#1", "P1001")
            if data:
                print("Decrypted record:", json.dumps(data, indent=2))
            print()

            print("=== Access attempt 2: wrong password ===")
            view_patient_record(fernet, "dr_asha", "WrongPassword", "P1001")
            print()

            print("=== Access attempt 3: unauthorized role (receptionist) ===")
            view_patient_record(fernet, "front_desk", "Reception#22", "P1001")
            print()

            print("=== Access attempt 4: unknown user ===")
            view_patient_record(fernet, "hacker99", "guess", "P1001")
```

Execution Output

Running the script produces the following console output, demonstrating each stage of the requirement:

```
 === Setting up users ===
Registered: dr_asha (doctor), front_desk (receptionist)

=== Adding patient records (encrypted before storage) ===
[STORED] Patient P1001 added — data encrypted at rest.
[STORED] Patient P1002 added — data encrypted at rest.

[RAW DB CONTENT] patient_id=P1001 ->
gAAAAABqg9FNfdZc70qEV2sIAxwradAE3zwbYVQrJxd2rEpASoHPLlH3yonw...  (truncated ciphertext)
```

```
=== Access attempt 1: authorized doctor ===
[AUTHORIZED ACCESS] dr_asha (doctor) decrypted record for patient P1001.
Decrypted record: {
  "name": "Ramesh Kumar",
  "diagnosis": "Type 2 Diabetes",
  "prescription": "Metformin 500mg twice daily",
  "timestamp": "2026-08-18T03:28:13.579144"
}

=== Access attempt 2: wrong password ===
[DENIED] Authentication failed for 'dr_asha'.

=== Access attempt 3: unauthorized role (receptionist) ===
[DENIED] User 'front_desk' (role: receptionist) is not authorized to view decrypted
diagnosis/prescription data.

=== Access attempt 4: unknown user ===
[DENIED] Authentication failed for 'hacker99'.
```

Analysis: How Confidentiality Is Maintained

1. **Data at rest is never plaintext.** The `[RAW DB CONTENT]` output shows that even direct access to the storage file (`patient_db.json`) reveals only Fernet ciphertext (`gAAAAAB...`) — an attacker who steals the database file learns nothing about any patient's name, diagnosis, or prescription without the separate encryption key.
2. **Decryption is gated behind authentication.** `view_patient_record()` calls `authenticate()` first; a wrong password (attempt 2) or unknown user (attempt 4) is rejected *before* any decryption is attempted — the plaintext is never even computed for a failed login.
3. **Decryption is further gated behind authorization.** Even a correctly authenticated user (`front_desk`, attempt 3) is denied if their role isn't permitted to view clinical data — demonstrating that authentication alone is not treated as sufficient; role-based access control enforces the principle of least privilege on top of it.
4. **Integrity is protected as a side effect of using Fernet.** Fernet's HMAC verification means any tampering with the stored ciphertext (e.g., an attacker modifying the database file directly) would cause decryption to fail safely with an error, rather than silently returning corrupted or attacker-controlled data.
5. **Passwords themselves are never stored or logged in plaintext** — only a salted PBKDF2 hash is kept, so even a breach of `users.json` does not directly expose user credentials.

Conclusion: The application satisfies the problem statement's requirement — patient ID, name, diagnosis, and prescription are encrypted before storage, and decryption is only ever performed for users who both authenticate successfully *and* hold an authorized role, maintaining confidentiality of patient data at every stage: at rest, during access control, and during retrieval.